Clasificación automática de 104 tipos de flores con redes neuronales profundas*

Angel Eduardo-Luna[a,1], Andrea Estefanía De la Peña Contreras[a,2], Jannette Cristina Flores Rodríguez[a,3], Rey Iván de la Fuente Chávez[a,4] and Andrés Huerta Robinson[a,5]

[a]*Escuela de Ingeniería y Ciencias, Tecnológico de Monterrey, Monterrey, Nuevo León, México*

INFORMACIÓN

Palabras clave:
Clasificación de imágenes
Redes neuronales convolucionales
Aprendizaje profundo
F1 macro
Kaggle

Resumen

Este trabajo aborda el problema de clasificación automática de imágenes de flores utilizando técnicas de aprendizaje automático y redes neuronales profundas. Se utiliza el conjunto de datos de la competencia *Petals to the Metal – Flower Classification on TPU* de Kaggle, compuesto por 23,847 imágenes RGB distribuidas entre 104 clases distintas de flores. El conjunto presenta variaciones importantes en tamaño, color, iluminación, orientación y composición de las imágenes, además de un desbalance en la cantidad de ejemplos disponibles para cada clase.

La tarea consiste en desarrollar un modelo capaz de identificar correctamente la clase de una flor a partir de una imagen. Debido a la similitud visual entre algunas categorías y a la variabilidad dentro de una misma clase, se considera principalmente el uso de redes neuronales convolucionales (CNN), ya que permiten extraer automáticamente características relevantes como formas, bordes, colores y patrones visuales. También se consideran arquitecturas previamente desarrolladas, como ResNet, AlexNet e Inception, así como métodos tradicionales de aprendizaje automático, entre ellos KNN, bosques aleatorios y máquinas de vectores de soporte, con el objetivo de comparar distintas alternativas para la solución del problema.

Para evaluar el desempeño se considera especialmente la métrica F1 macro utilizada por Kaggle, ya que asigna la misma importancia a cada una de las 104 clases independientemente de la cantidad de imágenes disponibles. De esta forma, el proyecto busca seleccionar y posteriormente implementar una estrategia de clasificación que logre un buen desempeño general sin favorecer únicamente a las clases con mayor cantidad de ejemplos.

1. Contexto del problema

La diversidad del mundo natural es enorme: se estima que existen más de 5,000 especies de mamíferos, 10,000 de aves y 30,000 de peces, pero aún mayor, hay más de 400,000 tipos de flores ("Petals to the Metal - Flower Classification on TPU — kaggle.com", s.f.). Distinguir entre ellas llega a requerir conocimiento especializado y, aún para una persona con experiencia, muchas flores son visualmente muy parecidas entre sí. Identificarlas de forma manual es lento y no es escalable cuando se tienen miles de fotografías. Por eso, resulta algo valioso poder contar con modelos que puedan reconocer el tipo de flor de forma automática a partir de una imagen.

Este proyecto parte de la competencia "Petals to the Metal", encontrado en Kaggle, cuyo reto es construir un modelo que identifique el tipo de flor en un conjunto de imágenes. La competencia se limita a 104 clases de flores, y consiste en un problema de clasificación multiclase de imágenes en el que el modelo debe tener la capacidad de identificar valores finos como la forma, estructura y color para distinguir entre categorías cercanas.

Este problema tiene varias dificultades que lo hacen relevante desde un enfoque de redes neuronales. Primeramente, las clases son heterogéneas, ya que algunas de ellas solo tienen un subtipo de flor y otras agrupan muchas subvariedades. Además, las imágenes de las flores no fueron capturadas en el mismo entorno, dando más variedad a las mismas. Por último, el número de imágenes por cada clase tampoco es uniforme, por lo que, considerando la métrica de evaluación, es un punto fuerte a tomar en cuenta que el modelo funcione de manera efectiva tanto para clases con pocos ejemplos como también para aquellas con muchos.

2. Descripción del dataset

2.1. Origen

Los datos vienen de *Petals to the Metal – Flower Classification on TPU*, una competencia *Getting Started* de Kaggle cuyo objetivo es aprender a entrenar modelos en TPU ("Petals to the Metal - Flower Classification on TPU — kaggle.com", s.f.). Según la descripción de la competencia, las imágenes se tomaron de cinco conjuntos de datos públicos. Algunas clases son muy específicas (por ejemplo, *pink primrose*, que contiene un solo subtipo de flor) y otras agrupan muchas subvariedades (por ejemplo, *wild rose*).

Al revisar la lista de clases nos dimos cuenta de que las primeras 102 siguen exactamente el orden del conjunto Oxford 102 Flowers (Nilsback y Zisserman, 2008): 94 conservan el mismo nombre y las otras 8 se renombraron, se corrigieron o se ampliaron (por ejemplo, *oxeye daisy* pasó a ser *daisy* y *bearded iris* pasó a ser *iris*). Las dos clases extra

*Situación problema: Metodología. Diseño de Redes Neuronales y Aprendizaje Profundo (TC2035.201). Basado en la competencia *Petals to the Metal – Flower Classification on TPU* (Kaggle).

[1]Matrícula: A00842657
[2]Matrícula: A00840973
[3]Matrícula: A01255711
[4]Matrícula: A01709445
[5]Matrícula: A00838626

Tabla 1
Particiones del conjunto de datos.

Partición	Imágenes	%	¿Etiquetado?
Entrenamiento	12,753	53.5	Sí
Validación	3,712	15.6	Sí
Prueba	7,382	31.0	No
Total	23,847	100.0	

son *common tulip* y *wild rose*. Las etiquetas están en inglés, así que usamos los nombres originales en cursiva.

2.2. Tamaño y estructura

En total hay 23,847 imágenes a color (RGB) de 104 clases, repartidas como muestra la Tabla 1. Vienen en formato TFRecord, que junta muchas imágenes en pocos archivos binarios para que la TPU las lea rápido. Cada registro tiene tres campos: `image` (la foto codificada en JPEG), `id` (un identificador único) y `class` (la etiqueta, un entero de 0 a 103). Los registros de prueba no traen `class`, porque esas etiquetas solo las tiene Kaggle para calificar.

Cada partición viene en 4 resoluciones: 192×192, 224×224, 331×331 y 512×512 píxeles, con 16 archivos TFRecord por partición, por lo que la base de datos trae 192 archivos TFRecord (4 resoluciones × 3 particiones × 16 archivos) más `sample_submission.csv`. Tener varias resoluciones nos conviene porque no todas las arquitecturas se diseñaron para el mismo tamaño de entrada: ResNet50, DenseNet201 y EfficientNetB0 hacen uso de 224 × 224, mientras que las EfficientNet más grandes usan resoluciones mayores. Para dimensionar el problema, una sola imágen de 224 × 224 ya tiene 150,528 valores.

2.3. Ejemplos de las imágenes

La tabla 2 muestra algunas clases concretas y lo que cada una nos dice sobre el problema. Además, dentro de una misma clase, las fotos cambian mucho de escala, pose e iluminación (Nilsback & Zisserman, 2008)

La Figura 1 muestra fotos reales del conjunto de entrenamiento.

2.4. Balance de clases y métrica de evaluación

Con 12,753 imágenes de entrenamiento y 104 clases, tenemos en promedio 123 imágenes por clase ($12{,}753/104 \approx 122.6$). Sin embargo, las clases no tienen el mismo tamaño: como unas clases agrupan muchas subvariedades y otras solo una, algunas tienen muchas más imágenes que otras.

Por este desbalance, la exactitud no basta: un modelo puede acertar mucho en las clases grandes, fallar en las clases pequeñas y aún verse bien. Por esto mismo, Kaggle califica con el F1 macro ("Petals to the Metal - Flower Classification on TPU — kaggle.com", s.f.). Para cada clase c se calcula la precisión P_c, la sensibilidad (*recall*) R_c y su media armónica:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \tag{1}$$

$$F1_c = \frac{2\,P_c\,R_c}{P_c + R_c}, \qquad F1_{\text{macro}} = \frac{1}{104}\sum_{c=0}^{103} F1_c, \tag{2}$$

donde TP_c, FP_c y FN_c son los verdaderos positivos, falsos positivos, y falsos negativos de la clase c . Así todas las clases pesan lo mismo, sin importar cuántas imágenes tengan.

2.5. Implicaciones para el proyecto

Como Kaggle ya separa entrenamiento y validación, no necesitamos hacer nuestra propia partición: entrenaremos con `train`, elegiremos modelos e hiperparámetros con `val` y usaremos `test` solo para la predicción final. Las imagenes vienen en JPEG con valores de 0 a 255, así que habrá que decodificarlas y escalarlas (a [0, 1] o con el preprocesamiento que pida cada red preentrenada). Como las etiquetas son enteros, la red producirá 104 valores de salida, uno por cada clase, que podrán transformarse en probabilidades mediante Softmax. El entrenamiento se realizará utilizando entropía cruzada categórica dispersa.

3. Algoritmos y aplicaciones

Dado el problema planteado, su solucion implica el uso de algoritmos de vision por computadora.

3.1. Redes Neuronales Convolucionales (CNNs)

Las redes neuronales convolucionales (CNN) son una alternativa especialmente adecuada para este problema debido a su capacidad para procesar información espacial. Mediante filtros convolucionales pueden aprender automáticamente características de las imágenes, como bordes, texturas y formas. En las capas internas pueden emplearse funciones de activación como ReLU, mientras que Softmax puede utilizarse en la salida para representar las probabilidades de las distintas clases.

Este método parece ser el principal y el mejor para producir resultados optimos, aunque, dado a la gran cantidad de datos que se tienen que tratar y la variedad de operaciones que se llevan a cabo, hay que considerar que es un metodo lento y computacionalmente pesado. Esto habilita la posibilidad de usar otros algoritmos que se ajusten a la necesidad del problema.

3.2. Otros algoritmos sin aprendizaje profundo

k vecinos más cercanos (KNN) Cada imagen lo representa como un vector de características. Usa normalmente la distancia euclidiana para medir las similaridades entre los puntos, y compara con sus k vecinos más cercanos para clasificar la imagen. Es un algoritmo de aprendizaje supervisado.

Bosque aleatorio Se compone de un conjunto de árboles de decisión, se calcula la media de los resultados de los árboles.

Máquinas de Vectores de Soporte (SVMs) Este algoritmo encuentra la línea óptima para separar dos clases.

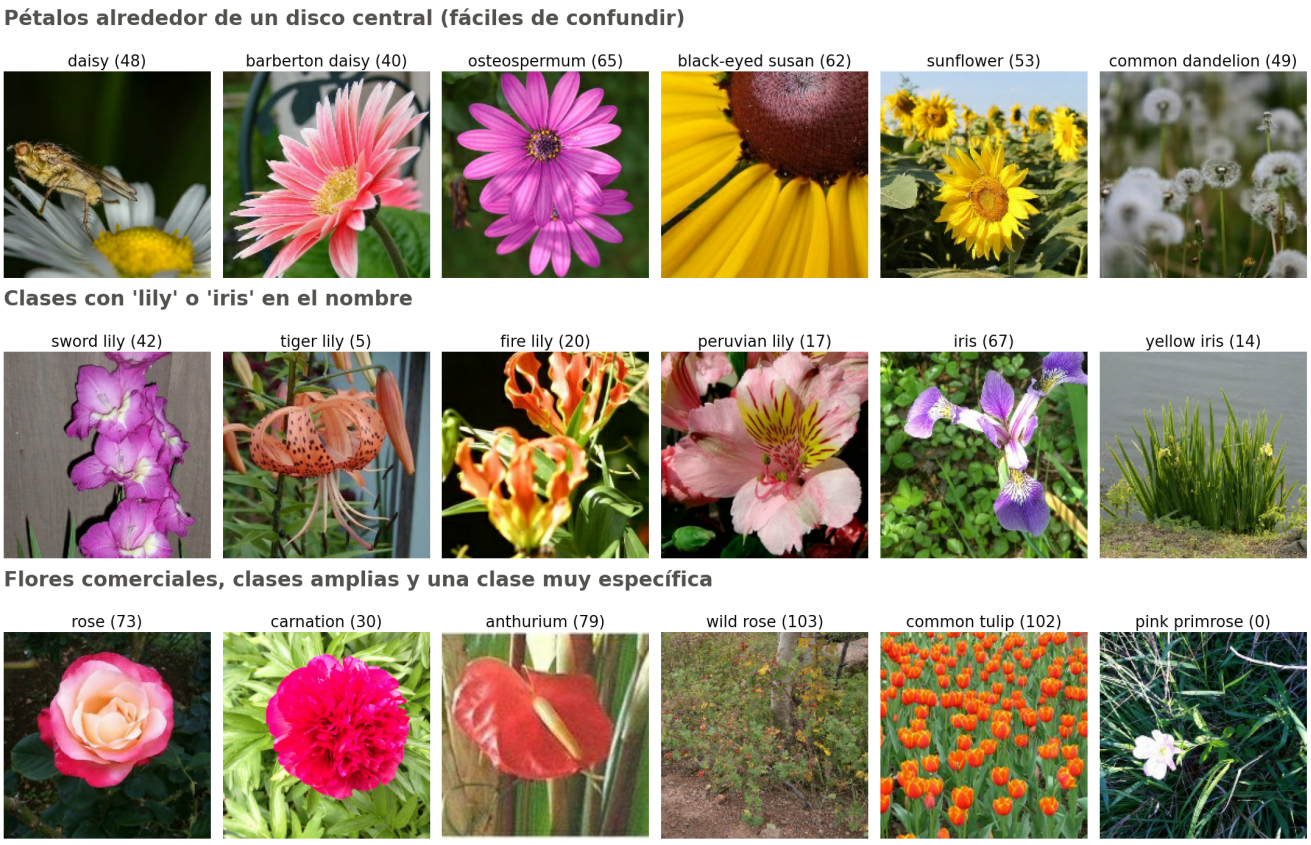


Figura 1: Ejemplos reales del conjunto de entrenamiento con su clase y número de etiqueta.

Tabla 2
Clases de ejemplo (nombre y etiqueta) más lo que ilustran.

Clase (etiqueta)	Que ilustra
sunflower (53)	Girasol. Flor grande y muy característica. Posiblemente sea de las más fáciles de identificar.
daisy (48), *barberton daisy* (40), *osteospermum* (65), *black-eyed susan* (62)	Margarita, gerbera, osteospermum y rudbeckia. Todas tienen pétalos alrededor de un disco central. Pueden surgir confusiones entre ellas.
sword lily (42), *tiger lily* (5), *fire lily* (20), *peruvian lily* (17)	Gladiola, lirio tigre, gloriosa y alstroemeria. Todas dicen *lily*, pero son de cuatro familias botánicas distintas: el modelo tiene que fijarse en la forma, no en el nombre.
iris (67), *yellow iris* (14)	Misma forma general, la diferencia principal es el color.
rose (73), *wild rose* (103)	Dos clases de rosas. *Wild rose* es amplia (varias subvariedades) y no viene de Oxford 102.
common tulip (102)	Tulipán. Clase amplia que tampoco viene de Oxford 102.
pink primrose (0)	Clase muy específica con un solo subtipo de flor.
carnation (30)	Clavel, una de las flores comerciales que se producen en el Estado de México.

Una variación llamada SVM multiclase existe para problemas multiclase, como este contexto.

3.3. Otros modelos de aprendizaje profundo

ResNet Conocida también como *Residual network*. Útil para entrenar redes neuronales muy profundas. La cota superior para implementar este método es la capacidad de nuestro *hardware* o acceso a equipo computacional que sea capaz de entrenar este tipo de redes más complejas.

Otros modelos ya implementados Como **AlexNet** o **GoogleNet/inception** que tienen una gran capacidad para reconocer y clasificar imágenes.("What Is Image Classification? | IBM — ibm.com", s.f.)

3.4. Aplicaciones posibles

Un modelo capaz de clasificar tipos de flores a partir de imágenes puede tener distintas aplicaciones reales. Una de ellas es la **identificación automática de plantas**, donde un

Tabla 3
Arquitectura propuesta de la red neuronal convolucional.

Capa	Configuración	Dimensión de salida
Entrada	Imagen RGB	$224 \times 224 \times 3$
Convolución 1	32 filtros de 3×3, padding *same* + ReLU	$224 \times 224 \times 32$
Max Pooling 1	Ventana de 2×2	$112 \times 112 \times 32$
Convolución 2	64 filtros de 3×3, padding *same* + ReLU	$112 \times 112 \times 64$
Max Pooling 2	Ventana de 2×2	$56 \times 56 \times 64$
Convolución 3	128 filtros de 3×3, padding *same* + ReLU	$56 \times 56 \times 128$
Max Pooling 3	Ventana de 2×2	$28 \times 28 \times 128$
Convolución 4	256 filtros de 3×3, padding *same* + ReLU	$28 \times 28 \times 256$
Max Pooling 4	Ventana de 2×2	$14 \times 14 \times 256$
Global Average Pooling	Promedio de cada mapa de características	256
Capa densa	256 neuronas + ReLU	256
Dropout	Tasa de 0.5	256
Salida	104 neuronas, una por clase	104

usuario podría tomar una fotografía y recibir como resultado la especie o clase más probable.

Un ejemplo de este tipo de aplicaciones yace en el apoyo a plataformas de ciencia ciudadana y el monitoreo de biodiversidad. Por ejemplo, iNaturalist ha acumulado más de 300 millones de observaciones hechas por personas comunes (iNaturalist, 2026) y utiliza un modelo de visión por computadora, entrenado con esas mismas observaciones, para sugerir la especia a partir de una fotografía. Sin este apoyo automático, cada observación dependería de que una persona la revisara detenidamente, lo que haría el proceso lento y difícil de escalar. Automatizar parte de la tarea ayuda a mejorar la eficiencia y aporta de gran manera a la obtención de datos sobre biodiversidad. Este proyecto es una versión a menor escala de este tipo de sistemas.

Otra aplicación sería como **herramienta educativa**, por ejemplo, en aplicaciones para estudiantes o visitantes de jardines botánicos que permitan reconocer flores y consultar información sobre ellas.

Sin embargo, el modelo desarrollado solo podría reconocer las 104 clases presentes en el conjunto de datos, por lo que una aplicación real más amplia requeriría entrenarlo con una mayor variedad de especies.

4. Metodología

4.1. Arquitectura de red propuesta

Para la clasificación de las 104 clases de flores se propone utilizar una red neuronal convolucional (CNN). La elección de este tipo de red se debe a que las CNN están diseñadas para trabajar con imágenes y permiten aprender automáticamente características espaciales relevantes, como bordes, texturas, colores y formas.

Las imágenes utilizadas tienen una resolución de 224×224 píxeles y tres canales de color RGB, por lo que la entrada de la red tendrá dimensiones $224 \times 224 \times 3$. A partir de esta entrada se utilizarán varios bloques convolucionales que aumentarán progresivamente el número de filtros mientras disminuyen el tamaño espacial de la representación.

La arquitectura propuesta es la siguiente:

Se utilizan filtros de 3×3 porque permiten analizar regiones pequeñas de la imagen y aprender patrones locales. Las primeras capas pueden aprender características simples, como bordes, cambios de color o texturas, mientras que las capas posteriores combinan estos patrones para representar estructuras más complejas de las flores.

El número de filtros aumenta de 32 a 256 conforme la información avanza por la red. Al mismo tiempo, las capas de *Max Pooling* reducen progresivamente el tamaño espacial de las representaciones. De esta manera, las primeras capas conservan mayor información sobre la posición de los elementos de la imagen, mientras que las capas profundas pueden almacenar una mayor cantidad de características aprendidas.

Después de los bloques convolucionales se utilizará una capa de *Global Average Pooling*, que resume cada mapa de características en un solo valor. Posteriormente se utilizará una capa completamente conectada de 256 neuronas y una capa de *Dropout* para reducir el riesgo de sobreajuste.

En las capas internas se utilizará la función de activación ReLU. Esta función introduce no linealidad en la red, permitiendo aprender relaciones más complejas que no podrían obtenerse utilizando únicamente transformaciones lineales.

Finalmente, la red tendrá una capa de salida de 104 neuronas, una por cada clase presente en el conjunto de datos. A partir de estos valores se podrá utilizar la función Softmax para representar la probabilidad asignada por el modelo a cada una de las clases.

4.2. Preprocesamiento de las imágenes

Antes de ingresar las imágenes a la red neuronal es necesario realizar una serie de pasos de preprocesamiento. Para este proyecto se utilizarán los archivos TFRecord correspondientes a la resolución de 224×224 píxeles. Cada registro contiene la imagen codificada en formato JPEG, su identificador y, para los conjuntos de entrenamiento y validación, una etiqueta entera entre 0 y 103.

Como primer paso, las imágenes serán extraídas de los archivos TFRecord y decodificadas como imágenes RGB de

tres canales. Debido a que el conjunto de datos ya proporciona una versión de 224×224 píxeles, no será necesario realizar un redimensionamiento adicional.

Los valores de los píxeles originalmente se encuentran entre 0 y 255. Para facilitar el entrenamiento, estos valores serán convertidos a tipo decimal y normalizados al intervalo $[0, 1]$ mediante

$$x_{\text{norm}} = \frac{x}{255}. \tag{3}$$

De esta forma, todas las entradas de la red se encuentran en una escala común y de menor magnitud.

Además, se propone utilizar aumento de datos (*data augmentation*) únicamente sobre el conjunto de entrenamiento. Se aplicarán transformaciones moderadas, como volteos horizontales, pequeñas rotaciones, acercamientos o alejamientos y variaciones leves de contraste. Estas transformaciones generan diferentes versiones de una misma imagen sin modificar su clase y ayudan a que el modelo no dependa de una orientación, posición o condición de iluminación específica.

El aumento de datos no será aplicado a los conjuntos de validación y prueba, ya que estos deben conservarse sin modificaciones aleatorias para poder evaluar el desempeño del modelo de manera consistente.

4.3. Función de pérdida y entrenamiento

El problema corresponde a una clasificación multiclase en la que cada imagen pertenece a una sola de las 104 clases posibles. Por esta razón se utilizará la entropía cruzada categórica dispersa (*sparse categorical cross-entropy*) como función de pérdida.

La red producirá 104 valores de salida, conocidos como *logits*, uno para cada clase. Estos valores pueden transformarse en probabilidades mediante la función Softmax:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{104} e^{z_j}}, \tag{4}$$

donde z_k representa el valor de salida asociado a la clase k y $\hat{y}_k$ representa la probabilidad estimada para dicha clase.

Para una imagen cuya clase correcta es c, la pérdida de entropía cruzada puede expresarse como

$$L = -\log(\hat{y}_c). \tag{5}$$

Esta función penaliza al modelo cuando asigna una probabilidad baja a la clase correcta. Si el modelo asigna una probabilidad alta a la clase verdadera, la pérdida será pequeña; en cambio, una predicción incorrecta realizada con alta confianza producirá una pérdida mayor.

Se utiliza la versión categórica dispersa debido a que las etiquetas del conjunto de datos se encuentran representadas directamente mediante números enteros entre 0 y 103, por lo que no es necesario transformarlas manualmente a vectores *one-hot*.

Durante el entrenamiento, primero se realizará una propagación hacia adelante (*forward propagation*) para obtener las predicciones y calcular la pérdida. Posteriormente se utilizará retropropagación (*backpropagation*) para calcular el gradiente de la pérdida con respecto a los parámetros de la red.

Para actualizar los parámetros se utilizará el optimizador Adam, el cual empleará los gradientes obtenidos mediante retropropagación. El entrenamiento se realizará utilizando minilotes de 32 imágenes, y el conjunto de entrenamiento será mezclado antes de formar cada lote.

Aunque la función de pérdida utilizada durante el entrenamiento será la entropía cruzada, el desempeño del modelo también será evaluado mediante F1 macro, debido al desbalance existente entre las 104 clases.

Referencias

iNaturalist. (2026). iNaturalist: A Community for Naturalists [[Accessed 28-09-2026]].

Nilsback, M.-E., & Zisserman, A. (2008). Automated Flower Classification over a Large Number of Classes. *2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing*, 722-729. https://doi.org/10.1109/ICVGIP.2008.47

Petals to the Metal - Flower Classification on TPU — kaggle.com [[Accessed 28-09-2026]]. (s.f.).

What Is Image Classification? | IBM — ibm.com [[Accessed 28-09-2026]]. (s.f.).

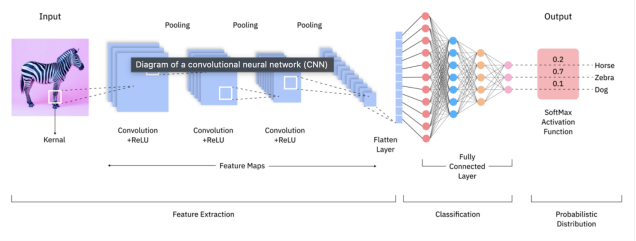


Figura 2: La convolución descompone la imagen en caracteristicas para aprender a clasificar ("What Is Image Classification? | IBM — ibm.com", s.f.).